FRED TALKS

6th August 2026

CONTENTS

Luddites and AI datacenters

SEANGOEDECKE.COM RSS FEED · 22 APR 2026 · SOURCE

Is it time to start burning down datacenters?

Some people think so. An Indianapolis city council member had his house recently shot up for supporting datacenters, and Sam Altman's home was firebombed (and then shot) shortly afterwards. People from all sides of the argument are sounding the alarm about imminent violence.

The obvious historical comparison is Luddism, the 19th-century phenomenon where English weavers and knitters destroyed the machines that were automating their work, and (in some cases) killed the machines' owners. Anti-AI people are reclaiming the term to describe themselves, and many of the leading lights of the anti-AI movement (like Brian Merchant or Gavin Mueller) have written books arguing more or less that the Luddites were right, and we ought to follow their example in order to resist AI automation[1].

Like many people, I have heard a lot about Luddism and Luddites, but only in the context of it being a general term for someone who is anti-technology. I was interested in learning more about the actual historical movement: what kind of people participated, what it was, and what it accomplished. I read Merchant's and Mueller's books, plus others[2], to try and figure all of this out. Who were the actual, historical Luddites? What can we learn from them about burning down datacenters?

Who were the Luddites?

The Luddites were a decentralized movement of artisans in the 1810s who engaged in violent protest - smashing machines, threatening violence, and ultimately killing people - over the fact that their jobs were being automated away. They were not rich, but they were certainly not unskilled labor: these were people who had apprenticed for seven years. They were mostly working from home, producing cloth from raw material given to them by their employer, often with tools rented from that same employer. They were working short weeks (three days, per William Gardiner) at their own discretion.

In the early 1800s, their skilled labor was becoming unnecessary. With the help of expensive machines, unskilled labor could now produce lower-quality cloth, so employers were beginning to pass over these artisans in favor of cheaper employees: children, unapprenticed workers, and

women[3]. Combined with the bad economic position of England at the time (at war with France, and thus deliberately cutting off much European trade), times were beginning to be very tough indeed. Starvation was a real threat.

What did they do?

Cloth artisans were groups of capable men who were used to getting their own way, knew each other very well, and were broadly respected in their communities. It was thus a natural response for them to organize into what was effectively a militant union. The Luddites would send anonymous threatening letters to their old (or current) bosses, warning them to stop using their machines. If they didn't comply, they would raid the workshop or factory, smashing the machines up.

They typically did not harm people, though they certainly delivered threats of bodily harm or even murder, and the raids were violent enough (e.g. shooting through windows) to have risked accidental deaths. In at least two instances where a factory owner was seen as unusually cruel, the Luddites did attempt assassinations: one unsuccessful, and one successful one that eventually prompted a crackdown that ended the movement for good.

Luddism was fully decentralized. Different communities could and did decide to engage in machine-raiding independently, particularly when news spread of the tactic succeeding. Although each community had its own influential men, there was never a single "leader of Luddism". King Ludd himself was a folk-tale figure. This made it an absolute nightmare for the British government to try and suppress them: putting down one Luddist group did nothing to prevent other groups from continuing to operate.

All the king's spies

I was surprised by how *difficult* it was for the government to get a hold of any of the local Luddist ringleaders. The government was willing to offer huge rewards to informers: at one point up to 40x the yearly wage. However, there were no takers for several years. Armies of spies were recruited and tasked with infiltrating Luddist groups, with absolutely no success.

Why was it so hard? Firstly, because the working class was so overwhelmingly pro-Luddist. People universally blamed the economic situation on the government and the factory owners (rightfully so, since the government had chosen to go to war and the factory owners had chosen to embrace automation). Secondly, the communities in question were so insular and tightly-knit that informers would have to rat on their friends and relatives. The handful of people who did eventually inform lived out the rest of their lives as pariahs.

Because each group was so insular, any spies trying to infiltrate the movement would have been complete strangers to the community, and would thus have a very hard time gaining the trust of a group of men who had known each other for their whole lives. The spies that did exist were restricted to the occasional inter-group Luddist meetings, where people didn't all know each other so closely. But it's unclear how important those meetings were, since Luddist groups didn't need to coordinate to achieve their goals. According to Merchant, the spies spent much of their time embellishing tales of an imminent revolution to encourage their employers to keep the money flowing.

The crackdown

In the absence of reliable information, the British government was forced to use force. And they did, sending 12,000 troops[4] into the northern counties. This served mainly as an intimidation tactic, since there was no standing Luddite army to fight, and the soldiers spent most of their time marching back and forth or being abused by the townspeople.

More successful was the imposition of a full police state in Yorkshire, under the magistrate Joseph Radcliffe, who was empowered to randomly grab people off the street and interrogate them for days. That pressure eventually convinced a handful of people to give up their local Luddist organizers, who were tried and inevitably hanged. Their deaths (and the ensuing climate of fear) ended the high-water mark of Luddist activity. Even then, Luddist raids continued on and off for *six more years* before petering out.

Did the Luddites succeed?

This is a tricky question. In one sense the answer is obviously no: the movement was crushed, many of their leaders were executed, the textile industry continued to be automated, and today there are no longer thousands of jobs for skilled British weavers, knitters, spinners and dyers. The pro-automation side won.

However, they did achieve a number of short-lived victories. Their early threats often succeeded in preventing the building of a factory in a particular location, or in delaying the adoption of industrial machinery in a particular shop by years. In one case, hosiers that had been spooked by Luddite activity gave out pre-emptive bonuses to their workers to discourage them from smashing up their machines (which were indeed not smashed).

The Luddites also scared the hell out of the British government, who (encouraged by their over-eager spies) thought they might have a genuine revolution on their hands. While they didn't get many legal concessions at the time, the specter of Luddism must have loomed over the labor reform movement of the 1800s, which saw the first anti-child-labor laws and the beginnings of independent inspection of factories.

Finally, every book I read argued that the Luddism movement may have created the first idea of a "working class", by unifying many previously-independent groups of workers against a common enemy. Seen this way, the "political arm"[5] of Luddism can arguably claim partial credit for every labor victory since the 1800s (though the ringleaders were still hanged and the weavers did still lose their jobs).

The Luddist approach in a nutshell

We can now describe the "Luddist approach" to fighting technological change:

1. Find a few conspirators in your existing community who agree with your political project (but don't join a broader organization, since that leaves you vulnerable)
2. Make public anonymous demands in support of your specific goals, backed up by threats of violence, signed by a fictional character that's easy for other groups to appropriate
3. If your threats are ignored, attack the physical machines in the dead of night, destroying them and threatening (but not killing) any guards
4. Hope your example inspires many more people to independently do (1)-(3) themselves
5. Keep raiding, optionally escalating to assassination of some of the bosses, until you bait a totalitarian crackdown from the government
6. Eventually get arrested and executed, to great public dismay
7. Twenty years later, your example inspires the first national trade unions

Note that starting or joining a national movement is *not* the Luddist approach. Staying almost entirely isolated in small cells helped the Luddists avoid government spies and made them impossible to root out without enforcing a police state. Note also that you need a *lot* of public support for this to work: so that you get a lot of copycat groups without having to explicitly organize them, and so that your property destruction and murder is taken sympathetically instead of getting you immediately reported and arrested.

Why Luddism is not a good model for the anti-AI movement

There are many reasons why this doesn't map onto the current anti-AI movement. First, Luddism grew from a homogeneous group of high-status workers whose jobs almost vanished overnight, not a broad group of people whose jobs are getting slightly worse because of AI (like the gig-economy workers Merchant endlessly references). That meant that Luddites had *really specific* asks: higher wages for piecework, a phased introduction of specific textiles machinery, and so on. They were not generally demanding that the machines all be immediately destroyed[6].

Second, Luddism was very local. A pre-existing group of artisans in a particular town would gather in that town - either at work or an inn, say - and decide to petition or raid the businesses in that town that were harming their livelihoods. AI concerns are not like this. It isn't businesses in Chicago or Tokyo that are making decisions that imperil Chicago's or Tokyo's jobs, it's businesses in San Francisco. Unlike the Luddists, anti-AI activists can't naturally organize with people they already know to take direct action where they already live.

Third, Luddist *victory* could also be local. If you successfully lobby your local cloth business to not use a weaving machine, you have secured your job at that business for a while. But if you successfully lobby your town (or even your country!) to not build a datacenter, it doesn't meaningfully improve your local position, since your job can be as easily replaced by a datacenter on the other side of the planet.

A total failure of leadership

Reading through the history of the Luddites from a modern perspective, I was struck by the near-total absence of *good government*. The artisans were left to work out their grievances with their bosses more or less by themselves, with no formal channels for complaint or any attempt at mediation. When the government did intervene - in response to near-universal unrest in *half of the country* - they did this:

1. Make machine-breaking and oath-taking capital crimes
2. Dump thousands of soldiers more or less at random into the area, with no plan to guard factories or do anything beyond just hang around in case a revolution broke out
3. Empower a single magistrate to arrest and interrogate whoever he wanted in order to root out the conspiracy

I suppose it worked, in the sense that it eventually succeeded in stopping the Luddist raids. But I can't help but think that even a token gesture of compromise (say, requiring employers to make their wages public, or restricting the most cheap-and-nasty factory-made textile products) would have gone a long way towards calming things down. This almost actually happened! The 1812 Framework Knitters' Bill, which had these provisions in it, passed the House of Commons but was shot down in the House of Lords.

Why did the government fail to even make a token attempt at compromise? Before the industrial revolution, I wonder if the workers and bosses of the English textiles industry were genuinely able to often just work out their problems together, so the government never really needed to do large-scale mediation. When that changed - when automation first made it possible for the bosses to durably "win" - government took a long time to realize, so there were some unpleasant decades of disempowered workers trying to bully factory-owners (via riots and death threats), and factory-owners trying to brutalize workers (via direct violence and automation).

Final thoughts

The Luddites weren't anti-technology in general. Instead, they were against four or five specific machines that were automating away their skilled work. Contrary to many of the books I read, I think that's actually fairly well understood today. But what surprised me was how truly decentralized and widespread the movement was. Every town with weavers faced the same incentives (the bosses to industrialize, and the workers to fight back) which created Luddism locally. And *nobody* was willing to report the Luddites: not for years, or for what would have been a fortune in cash, or in disagreement with their actions. They were only stopped by a truly fascist crackdown, with the army in the streets and the secret police pulling away random citizens for arrest and questioning.

I can see why modern "Luddites" - who are genuinely anti-technology - talk so much about the legacy of original Luddites. Luddism was a grassroots organization which notched up some real short-term wins, enjoyed near-total support among the public, and didn't seem to be troubled by infighting at all[7]. If you're an anti-AI campaigner, I bet all of that sounds great[8]. But I'm not convinced that the neo-Luddites really are the inheritors of Luddism. A load-bearing feature of Luddism is that it was *local*: it didn't have manifestos, or leaders, or factions, or even much explicit ideology beyond the artisans' immediate practical concerns. These were local men striking back against the local factories harming their local jobs. That simply isn't the case with AI, where a datacenter in China can take my job in Australia.

If it isn't time to start burning down datacenters, what can we do? Well, workers today are better off than the Luddites were, because we stand on their shoulders. Unlike the Luddites, today's workers can all vote (and I suspect there will be no shortage of anti-AI candidates to vote for).

It's a much less exciting answer to say "try and get better laws passed" than to say "viva the revolution, where's my Molotov cocktail". Still, if the Luddites had that option, I suspect they would have used it.

1. I got linked this article calling AI a "fascist artifact" (on a blog called "Breaking Frames", a clear reference to Luddism) while I was writing this blog post.

   ↩

2. I really enjoyed Merchant's book and did not enjoy Mueller's (which I found to be 10% about the Luddites and 90% about interminable intra-Marxist ideological arguments). I also read The Luddites, which was effectively a dry summary of the ground Merchant covers, a bunch of other essays, and went back and forth with ChatGPT and Claude on some of the key questions.

   ↩

3. Merchant (around page 134) attempts to characterize Luddism as a pro-feminist movement, citing some examples of women helping organize raids, but later on even he (page 162) quotes a representative of the Irish weaver's guild effectively saying "we don't have your English problems of women working in the industry". In general it's a bit frustrating that the popular books on Luddism are all fairly uncritically pro-Luddist (though not surprising, I suppose). Merchant doesn't touch at all on the Luddist practice of going around to knitting-shops with women and "discharging them from working".

   ↩

4. Sometimes this is described as more troops that were sent to fight Napoleon (even by Merchant himself on page 89), but that isn't right.

   ↩

5. In quotes because it was not an official Luddist group (there were none), just people who were trying to stop the violence through lobbying and legislation.

   ↩

6. Otherwise why would any boss agree, instead of just waiting for the Luddites to do it themselves?

   ↩

7. As far as I can tell this is true: the Luddists basically had no internal conflict. I think this is because each individual cell knew each other well already, and so handled their disagreements privately (instead of by writing pamphlets), and disagreements between cells didn't matter that much because they had no need to coordinate.

   ↩

8. It beats the hell of the other popular reference, Dune's Butlerian Jihad, which was two generations of brutal violence followed by the reimposition of the feudal system. (Although, at least the Butlerian Jihad *succeeded*…)

   ↩

Search has its own bitter lesson

DOUG TURNBULL · 22 JUN 2026 · SOURCE

An agent with grep does quite well at search.

It's shocking to search technologists. We want to cocoon the problem in technology. We care about algorithms, knowledge graphs, ranking: ever-and-ever smarter retrieval.

It turns out, though, you can play a bit of a trick. If you convince everyone to optimize content for your search engine, you'll have built the best search engine.

There's Sutton's bitter lesson about unleashing raw compute on a problem. But there's a different bitter lesson in search: algorithms matter less than convincing the world to optimize for your search engine.

Sure, technology acts as the trigger. In Web Search, Google shifted away from text relevance to prioritize authoritativeness (PageRank). Their success told the market: create content others want to link to. They reshaped the Web for good (or ill) around that incentive.

Google crowdsourced search quality to the Web. Claude Code outsources it to every developer.

Developers depend on agents to find documentation on the file system. We like that dopamine hit. That thrill of Claude Code knowing exactly how to reason about our code.

When agents make boneheaded mistakes, we stress. We frantically update documentation, we link to it from AGENTS.md, we add a new document, we place it in a useful location. We do anything - absolutely anything - to ensure agents know how to find and understand documentation.

Just as the SEO expert reads the tea leaves of Google's algorithm. The software developer writes to the LLM to make documentation findable + useful.

A search system with these incentives overcomes technologically sophisticated systems. For example, classic RAG rests on a foundation of technically sophisticated question answering. Answers often look like conversationally valid responses, but with little linkage back to questions it might anser:

Question:

> What is a runner test?

Answer:

> Verifies that small-dollar deposits and withdrawals can be executed against real test bank accounts during account reconciliation.

Yes that's literally an answer to the question.

But I wouldn't consider this "good documentation". It's not anchored to any concepts from the question. It's just an out of context bit of text.

If you wrote this for a coding agent, you'd probably organize the chunk to give hints about why it's useful.

> docs/tests/runner.md
>
> # Runner Tests
>
> ## Purpose
>
> We need to ensure the system runs end to end
>
> ## Usage
>
> Use this style of test sparingly. Unit and standard e2e tests should usually be preferred.

> ## What is it
>
> A runner test actually interacts with real-life test bank accounts to ensure we can correctly deposit / withdraw money and that the full banking system works.

That's a far more descriptive piece of content to an LLM. Being encouraged to write *for the LLM itself* makes all the difference.

(As an aside, LLMs see candidate *answers* in their context before the user's question - so just treating RAG as question answering doesn't work).

What often matters isn't algorithms. It's programming the content author to create useful LLM context. That moves content discovery from random, disconnected factoids to documentation connecting information to problems to solve.

You learn that the trick to RAG isn't creating good question answering systems. Its writing better answers.

The enterprise search morass… is over?

Compare how coding agents program us to the eternal nightmare of your company's wiki.

When people write in the wiki, they do it for themselves. Nobody is incentivized to make information findable by anyone. No author sits obsessing how to 'SEO' the content for Confluence search.

If someone is 'lucky' enough to find your article, they can barely understand it. It's written in your team's jargon. And it's probably out of date anyways.

For these reasons, I always tell people they'd do better to write a public blog article. Blogs assume zero context. They're clearly timestamped as an artifact of a moment in time. To avoid shame or promote your career, you're incentivized to create something useful, interesting, and findable.

Writing for a coding agent has the same property. You're writing documentation for a targeted reader. You care to SEO your content for Claude Code + grep.

Enterprise search fails because there's no feedback loop between authorship and findability. Coding agents solve that, finally giving some hope to organizing company information into findable knowledge.

The real outcome of coding agents isn't just code generation. It's closing the feedback loop between company context and the next piece of work being done. That old fable of Enterprise Search - improving workforce productivity by leveraging existing insight - can finally become true.

Feedback to content authors drives user relevance

Look around your system. Who creates the content? What's in it for them to make content findable?

A recurring theme in my consulting looks something like this - a job search company notices spammy behavior from job posters. They keyword stuff every programming language in some hidden text element. The usual reaction: that's spammy and we should ban them.

That's too black and white. You have content creators willing to put in work. Shift their black-hat SEO over to an acceptable white-hat SEO. Document what's acceptable. What's considered malicious.

Focus on feedback. Make the content-optimization path rewarding. Give creators feedback+guidance during creation. LLMs help detect spam in real-time. They push content creators to author what's useful not malicious.

Give content creators feedback. Once you've identified what 'good' is, give those content authors dopamine hits. Tell content creators what queries perform well for the content. Make it real-time, sticky, fun. Incentivize and reward behaviors you want - and punish those you don't.

Steer content creators towards the content you want. Don't hope algorithms can overcome drivel and slop.

Your search engine isn't a bundle of algorithms ranking for users. It's a relationship between authors and users. That bitter lesson puts grep above the smartest reranker and embedding model. Forget it at your peril.

Enjoy softwaredoug in training course form!

Starting May 18!

Signup here - http://maven.com/softwaredoug/cheat-at-search

I hope you join me at Cheat at Search with Agents to learn use agents in search. build better RAG and use LLMs in query understanding.

Headless everything for personal AI

INTERCONNECTED · 18 APR 2026 · SOURCE

It's pretty clear that apps and services are all going to have to go *headless:* that is, they will have to provide access and tools for personal AI agents without any of the visual UI that us humans use today.

By services I mean things like: getting a new passport; finding and booking a hotel or a flight; managing your bank account; shopping for t-shirts with a minimum cotton weight from brands similar to ones that you've bought from before.

Why? Because using personal AIs is a better experience for users than using services directly (honestly); and headless services are quicker and more dependable for the personal AIs than having them click round a GUI with a bot-controlled mouse.

Where this leaves design for the services… well, I have some thoughts on that.

Headless services? They're happening already.

Already there is MCP, as previously discussed (2025), a kind of web API specifically for AI. For instance, best-in-class call transcriber app Granola recently released their MCP and now you can ask your Claude to pull out the meeting actions and go trawling in your personal docs to find answers to all the question. A good integration.

But command-line tools are growing in popularity, although they used to be just for developers. So you can now create a spreadsheet by typing in your terminal:

```
gws sheets spreadsheets create --json '{"properties": {"title": "Q1 Budget"}}'
```

Here are some recently launched CLI tools:

- gws: Google Workspace CLI, Drive, Gmail, Calendar, and every Workspace API. Zero boilerplate. Structured JSON output. 40+ agent skills included.
- Obsidian CLI to do anything you can do with the (very popular) note-taking app - like keep daily notes, track and cross off tasks, search - from the CLI
- Salesforce CLI and, look, I don't really understand what the Salesforce business operating system does, but the fact it has a CLI too is significant.

And here is CLI-Anything (31k stars on GitHub) which auto-generates CLIs for ANY codebase.

Why CLIs?

It turns out that the best place for personal AIs to run is on a computer. Maybe a virtual computer in the cloud, but ideally your computer. That way they can see the docs that you can see, and use the tools that you can use, and so what they want is not APIs (which connect webservers) but little apps they can use directly. CLI tools are the perfect little apps.

CLIs are composable so they are a better fit for what users actually do.

By composable I mean you can: query your notes then jump to a spreadsheet then research the web then jump back to the spreadsheet then text the user a clarifying question then double-check your notes, all by bouncing between CLIs in the same session.

A while back app design got obsessed with "user journeys." Like the journey of a user finding a hotel then booking a hotel then staying in it and leaving a review.

But users don't live in "journeys." They multitask; they talk to people and come back to things; they're idiosyncratic Try grabbing the search results from the Airbnb app and popping them in a message to chat on the family WhatsApp, then coming back to it two days later. It's a pain and you have to use screenshots because apps and their user journeys are not composable.

CLIs are composable because they came originally from Unix and that is the Unix tools philosophy: tools were designed so that they could operate together.

Personal AIs like Openclaw or [Poke](https://poke.com do what users want and don't follow "designed" user journeys, and as a result the composed experience is more personal and way better. CLIs are a great underlying technology for that.

CLIs are smaller than regular apps and so they are easier to secure.

The alternative to providing special tools for AIs is that the AIs use the same browser-based apps that we do, and that's a terrifying prospect.

For one, AIs are really good at finding security holes. The new Mythos model from Anthropic is so good at discovering security flaws that it has been held back from the public and governments are convening emergency meetings of the biggest banks.

And including a UI increasing complexity and makes security holes more likely.

Here's a recent shocking example. Companies House is the national register of all companies, directors, and accounts for England and Wales. Users could view *and edit* any other user's account:

> a logged-in company director could exploit the flaw by starting from their own dashboard and then trying to log into another company's account.
>
> Once they reach the 2FA block, which they would not be able to pass, all that was required was to click the browser's back button a few times. Typically, the user would be taken back to their own dashboard, but the bug instead returned them to the company they had tried to log into but couldn't.

This bug had been present since October 2025.

Imagine a future where personal AIs are filing company records, and one of them notices this security hole overnight and posts about it on moltbook or whatever agent-only social network is most popular. The other agents would exploit the system up, down and sideways before the engineering team woke up.

The only viable solution is that services need to security hardened, and to do that they need to be simplifying and minimised. Again, CLI tools are a great fit.

What does this mean for front-end design?

Design won't go anywhere.

Sure, the front-end should be driving the same CLI tools that agents use.

Arguably it's more important than ever: human users will encounter services, figure out what they can do, and pick up their vibe from using the app, just as now.

Then they'll tell their personal AI about the service and never see the front-end again, or re-mix it into bespoke personal software.

So from a usability perspective I see front-end as somewhat sacrificial. AI agents will drive straight through it; users will encounter it only once or twice; it will be customised or personalised; all that work on optimising user journeys doesn't matter any more.

But from a vibe perspective, services are not fungible. e.g. if you're finding a restaurant then Yelp, Google Maps, Resy, and The Infatuation are all more-or-less equivalent for answering that question but clearly they are completely different and you'll use different services at different times.

Understanding that a service is *for you* is 50% an unconscious process - we call it brand - and I look forward to front-end design for apps and services optimising for brand rather than ease of use.

If I were a bank, I would be releasing a hardened CLI tool like *yesterday.*

There is so much to figure out:

- How do permissions work? Should the user get a notification from their phone app when the agent strays outside normal behaviour? How do I give it credentials to act on my behalf, and how long do they last?

- How does adjacency work? My bank gives me a current account in exchange for putting a "hey, get a loan!" button on the app home screen. How do you make offers to an agent?

Headless banks.

Headless government?

I'd love to show you a worked example here. I vibed up a suite of four CLI tools that wrap four different services from UK government departments.

If I were renting a house, I would set my agent to learning about the neighbourhoods using one of these tools. Another tool will be helpful next time I'm buying a used car. (There's a Companies House command-line tool too.)

But I won't show you the tools because I don't want to be responsible for maintaining and supporting them.

I wish Monzo came with an official CLI. I wish Booking.com came with a CLI. I reckon, give it a year, they will.

Auto-calculated kinda related posts:

If you enjoyed this post, please consider sharing it by email or on social media. Here's the link. Thanks, —Matt.

Why I don't like the "staff engineer archetypes"

SEANGOEDECKE.COM RSS FEED · 03 MAY 2026 · SOURCE

The most influential piece of writing about staff engineers in the last decade has to be Will Larson's *Staff engineer archetypes*. He argues that the "staff engineer" title covers at least four very different roles: the team lead, the architect, the solver, and the right hand. This taxonomy gets cited a lot as advice for people who are trying to become effective staff engineers. For both of my promotions to staff engineer, my manager at the time linked me to the "staff engineer archetypes" and asked me to consider which of these archetypes I was aiming towards.

These archetypes definitely exist[1]. However, I think it's bad practical advice to tell engineers to try and target them.

Archetypes do not make good goals

To see why, let's take the "team lead" archetype. Larson describes this as an informal technical leadership role: not necessarily an explicit authority figure, but someone who's good at scoping work, planning projects, and maintaining the kind of relationships (e.g. with other teams) needed to successfully ship. If you want to fill this role, shouldn't you start trying to do these things? No! You don't become a technical leader by trying really hard to be a technical leader, much like you don't become a writer by trying really hard "to be a writer". You become a technical leader by *doing good technical work* until your skills and relationships emerge organically.

I wrote about this process in *Ratchet effects determine engineer reputation at large companies*. To get good at shipping large complex projects, you must start by shipping tiny pieces of work, until you're familiar enough with the system and you've built enough trust to take on slightly larger pieces. At each stage, if you do good work - "good work" here means "deliver shareholder value" - you will very naturally be given opportunities to work on more complex and important things. If you try to jump ahead, you're going to run into all kinds of problems:

- Important projects are usually assigned top-down, not bottom-up, so you'll either be trying to muscle out the planned engineering lead for a project or to pitch your own (complex, important) engineering task to senior management. Either way, good luck with that!
- You likely won't have a good enough relationship with senior management to know what their real priorities are.
- If you're not yet trusted to execute, you may get assigned "minders" (often current staff engineers) who will ghost-lead the project through you[2].
- You'll likely make poor technical decisions.

The other archetypes are like this as well. If you want to become a successful architect, you do not get there by studying software architecture in the abstract, because you can't design software you don't work on. The "solver" and "right hand" archetypes both rely on having an enormous amount of trust and influence. You can't aim for those archetypes directly, because trust and influence accumulate over time. In fact, the idea of "aiming for" a particular staff engineer archetype reflects a misunderstanding of what the staff engineer role is. What is the defining attribute of the staff engineering role, then?

What is a staff engineer?

A staff engineer has to be useful to the company. Of course, a senior or mid-level software engineer ought to be useful too, but all they *have* to do is execute on the job in front of them. If they end up not providing value (maybe their project turns out to be unimportant, or they don't get the support needed to succeed) that's their manager's problem, not theirs[3]. In contrast, staff engineers are expected to deliver value regardless: to make the project work, or to find something else useful to do if the project truly can't be salvaged.

This is an unfair expectation. Often projects really do fail through no fault of your own, and sometimes it just isn't possible to conjure useful work from thin air. That's actually by design: **the staff engineer role is supposed to be unfair**. Something many engineers don't realize is that all senior management and executive leadership roles are unfair too, in the same way. That's

just part of the deal: executives are given power and great compensation, and in return they get thrown off the boat in bad weather[4]. “Staff engineer” is the first engineering role where you are held largely responsible for outcomes you don’t control.

Developing a “staff engineer mindset” thus has very little to do with the archetypes. Instead, you should:

1. Develop the habit of constantly asking yourself “is this useful to the company” (and answering correctly).

2. Lose the habit of worrying about if you’re being treated “fairly”. Instead, try to think about your role in terms of incentives and consequences.

At the beginning, you won’t look much like any of the staff engineer archetypes. You will look like being a level-headed engineer who can be trusted to move projects forward with a minimum of fuss, and who can be re-tasked to different work without complaining. You’ll also look like someone who’s paying a lot of attention to what their manager’s actual priorities are, and who is thinking hard about how to fulfil those priorities (instead of their own goals).

If you do this for long enough, you’ll eventually find yourself in one of the staff engineer archetypes. However, it probably won’t be the one you’re “aiming for”. The whole point of being a staff engineer is that you’re willing to fill whatever archetype the company needs at the time.

Final thoughts

In his original staff engineer post, Larson is pretty clear that these archetypes are more of an anthropological description of some of the varied niches staff engineers fill, not a how-to guide for succeeding in the role[5]. At the time, the “staff engineer” role was fairly new and people were still trying to figure out what it even meant. Pointing out that there were a few very different ways to succeed in the role was a genuinely novel observation.

The staff engineer archetypes are a good list of ways an engineer can be very useful to their organization - but only once they’ve built a deep relationship of trust with their organization’s leadership. Advice on how to succeed as a staff engineer should be about **how to build that trust**, not about what to do once you have it.

1. One caveat that is too pedantic for the body of the post: each tech company has a different structure of roles. Some don’t have the formal “staff” title at all, while others have “staff” as a fairly early rung on the ladder and a panoply of “senior staff”, “senior principal staff”, and

so on roles above it. Like all “staff engineer” discourse, this post is not about the word itself but about the point in the engineering job ladder where progression becomes significantly more difficult.

↩

2. Impressing your VP’s trusted lieutenants can actually be a good way to build trust in the medium-term, but you’d better hope you’ve built enough understanding of the system to do it right. If this process goes badly, your reputation in the org might be torched for years.

↩

3. In theory, at least. In practice it’s always better to be useful (again, in the sense of “delivering shareholder value”).

↩

4. This is why very senior leadership sometimes seem so unempathetic towards engineering complaints: their work environment operates by very different rules and norms to that of most engineers. I keep meaning to try and write about this and never succeeding. This draft is the closest thing I have to a deeper exploration of the point.

↩

5. For the record, my how-to guides are here and here.

↩